theorem_a3n.md

The Maximum Right-Isosceles-Free Subset of a Three-Row Board

Abstract

Let a(3,n) denote the largest number of lattice points that can be chosen from a board with 3 rows and n columns so that no three chosen points form a non-degenerate right isosceles triangle. We prove that

a(3,n) = n + 2 for every n ≥ 4.

The lower bound is an explicit construction valid for all n. The upper bound is proved by a potential-function argument on a finite automaton of 4-column states. The finite part of the upper bound is checked by computer, and the certificate is a short finite table that can be verified independently. The argument is therefore computer-assisted, though every reduction step is elementary.

1. Setting

A *board with 3 rows and n columns* is the set of lattice points (x, y) with $0 \le x \le n-1$ and $y \in \{0,1,2\}$. A set S of points is *admissible* if it contains no three points forming a non-degenerate right isosceles triangle, meaning a triple {A, B, C} with a right angle at some vertex V and equal legs VA = VB ≠ 0.

Column j of a set S is encoded as the 3-bit mask c_j = Σ_{y ∈ rows of S in column j} 2^y, with values in {0,...,7}. A sequence (c_0, ..., c_{n−1}) of masks determines S. We write pop(c) for the number of set bits, so the number of points in column j is pop(c_j).

2. Two elementary lemmas

Lemma 1 (horizontal extent). Any right isosceles triangle with vertices on the 3-row board has all its vertices within a horizontal range of at most 4 columns. More precisely, its x-coordinates span at most 4 consecutive integers.

Proof. Let the right angle be at P, and let the legs be u = (p, q) and w = ε(−q, p) with ε ∈ {+1, −1}, since w is a rotation of u by ±90°. The vertices, relative to P, have y-coordinates 0, q and εp. These lie within an interval of length at most 2 (the board has 3 rows), and that interval contains 0. Hence |q| ≤ 2 and |p| ≤ 2. The x-coordinates relative to P are 0, p and −εq, all lying in [−2, 2]. So the x-span is at most 4. □

Lemma 2 (locality). Let n ≥ 5. A set S on the 3×n board is admissible if and only if every set of five consecutive columns of S is admissible.

Proof. Forward direction: restriction of an admissible set to a sub-board is admissible. Backward direction: any triangle in S has x-span at most 4 by Lemma 1, so it lies inside some five consecutive columns. □

For n ≥ 5, Lemma 2 makes admissibility a *local* condition on windows of five columns, so the problem is a path problem in a finite graph. The graph is described next.

3. The state graph

A **state** is an admissible 4-tuple of masks s = (c_1, c_2, c_3, c_4), meaning that the 4-column board they encode is admissible. A **transition** s → t, where t = (c_2, c_3, c_4, c_5), is allowed when the 5-tuple (c_1, …, c_5) is admissible. The transition has **weight** pop(c_5), the number of points added in the new column. By Lemma 2, an admissible 3×n set with n ≥ 4 corresponds exactly to a path of n−3 states, where consecutive states differ by an allowed transition. The total number of points is

T = pop(c_1) + pop(c_2) + pop(c_3) + pop(c_4) + Σ over transitions of their weights.

Computationally there are 443 states and 1736 admissible 5-tuples.

4. Upper bound: a(3,n) ≤ n + 2

Proposition 3 (certificate). There is a function Φ from the set of states to the integers {0, 1, 2} with the following properties.

(a) Φ(s) ≥ 0 for every state s.
(b) For every allowed transition s → t with new column c_5: Φ(s) ≥ (pop(c_5) − 1) + Φ(t).
(c) For every state s: pop(c_1) + pop(c_2) + pop(c_3) + pop(c_4) + Φ(s) ≤ 6.

A concrete Φ is given in `phi3.json` (keys are states, values are Φ). It is computed as the longest-path potential: Φ(s) is the supremum over allowed paths starting at s of the sum of (weight − 1). This supremum is finite, which is the computational content of the claim that no cycle of the state graph has mean weight greater than 1.

Proof of the upper bound from Proposition 3. Let n ≥ 4 and let S be admissible on the 3×n board. Let s_1, …, s_k be its state path, where k = n − 3. Summing the inequality in (b) along the path gives

Σ over transitions (pop(new column) − 1) ≤ Φ(s_1) − Φ(s_k).

The total number of points is

T = [pop(c_1) + … + pop(c_4)] + Σ over transitions pop(new column) = [pop(c_1) + … + pop(c_4)] + (n − 4) + Σ over transitions (pop(new column) − 1) ≤ [pop(c_1) + … + pop(c_4) + Φ(s_1)] + (n − 4) − Φ(s_k) ≤ 6 + (n − 4) − 0 = n + 2,

using (c) for s_1 and (a) for s_k. □

Verification of Proposition 3. The conditions (a)–(c) were checked by two independent implementations: one that uses the DP window code, and one that uses only a brute-force test of all triples against the definition of a right isosceles triangle (`check_certificate.py`). Both confirm (a), (b) and (c) for all 443 states and all 1736 admissible 5-tuples. The maximum of pop-sum plus Φ over states is exactly 6, so the bound in (c) is tight for some state.

5. Lower bound: a(3,n) ≥ n + 2

Notation. Write the block B = (0, 0, 5, 5). Mask 5 means points in rows 0 and 2. The infinite periodic word (0,0,5,5)^∞ has 2 points in each of its 5s and none in its 0s, so each block of four columns contains four points.

Base sequences. For each b ∈ {8, 9, 10, 11}, the following 3×b column sequence is admissible, has b+2 points, and ends with the block (0,0,5,5):

- b = 8: (7, 0, 0, 7, 0, 0, 5, 5), with 10 points.
- b = 9: (7, 0, 2, 1, 5, 0, 0, 5, 5), with 11 points.
- b = 10: (3, 4, 1, 1, 4, 3, 0, 0, 5, 5), with 12 points.
- b = 11: (6, 1, 1, 1, 1, 4, 3, 0, 0, 5, 5), with 13 points.

For n = 4, 5, 6, 7, the following sequences are admissible with n+2 points:

- n = 4: (7, 0, 0, 7), with 6 points.
- n = 5: (7, 0, 2, 4, 5), with 7 points.
- n = 6: (7, 0, 2, 2, 0, 7), with 8 points.
- n = 7: (7, 0, 2, 4, 4, 4, 3), with 9 points.

Construction. For n ≥ 8, let b ∈ {8, 9, 10, 11} be the unique value with b ≡ n (mod 4). Form the sequence consisting of base B_b followed by (n − b)/4 copies of the block (0,0,5,5). This has n columns and b+2 + 4·(n−b)/4 = n+2 points.

Admissibility. By Lemma 2 it suffices to check the five-column windows.

- Windows lying inside the base are admissible, since the base is admissible.
- Windows lying inside the periodic part are windows of (0,0,5,5)^∞. There are four phases, namely (0,0,5,5,0), (0,5,5,0,0), (5,5,0,0,5) and (5,0,0,5,5). Each is admissible.
- Any window that is not of the first two kinds must start among the last four columns of the base. Those four columns are (0,0,5,5), which is the first four columns of the periodic word. So such a window is also a window of (0,0,5,5)^∞, and it is admissible.

Hence the constructed set is admissible. It has n+2 points. □

The construction was also checked directly, by brute force on all triples of points, for every n from 4 to 60 (`construct3.py`, `construct3.json`).

6. Conclusion

Combining Sections 4 and 5, for all n ≥ 4:

a(3,n) = n + 2.

For n ≤ 3 the values are a(3,1) = 3 and a(3,3) = 4 (computed).

7. Status and limitations

- **Hand-checked:** Lemma 1, Lemma 2, the upper-bound deduction from Proposition 3, all base sequences and the periodic argument for the lower bound.
- **Computer-assisted:** the existence and verification of the potential Φ in Proposition 3. This is a finite check on 443 states and 1736 transitions. We checked it with two independent codes. A reader who wants a purely human proof would need a closed-form potential, which we did not find.
- **Also computer-checked:** the lower-bound construction for n ≤ 60, and exact values up to n = 60 from the dynamic program, which agree with n+2 throughout.
- **Literature:** the only related source we found is the OEIS entry for the square-grid problem, A271906 (n ≤ 11), and a 2026 preprint on the square problem (Károlyi–Solymosi, arXiv:2607.22828). Neither contains this result. We did not carry out a complete literature search, so novelty is not guaranteed.

8. Files

All files are in `./`:

- `dp3.py`: window checks and the dynamic program for a(3,n).
- `potential3.py`: longest-path potential, D = 6.
- `phi3.json`: the potential table Φ.
- `check_certificate.py`: independent verification of Proposition 3.
- `lower3.py`, `construct3.py`, `construct3.json`: base sequences and the construction.